

```

        # put the rest of the other list into sortedResult.
        if iLeft == len(left):
            sortedResult.extend(right[iRight:])
            break
        elif iRight == len(right):
            sortedResult.extend(left[iLeft:])
            break

    print('The two halves merged into:', sortedResult)

    return sortedResult # Returns a sorted version of items.

myList = [2, 9, 8, 5, 3, 4, 7, 6]
myList = mergeSort(myList)
print(myList)

```

The *mergeSort.html* program contains the equivalent JavaScript program:

```

<script type="text/javascript">
function mergeSort(items) {
    document.write("<pre>" + ".....mergeSort() called on: [" +
        items.join(", ") + "]</pre>");

    // BASE CASE - Zero or one item is naturally sorted:
    if (items.length === 0 || items.length === 1) { // BASE CASE
        return items; ❶
    }

    // RECURSIVE CASE - Pass the left and right halves to mergeSort():
    // Round down if items doesn't divide in half evenly:
    let iMiddle = Math.floor(items.length / 2); ❷

    document.write("<pre>.....Split into: [" + items.slice(0, iMiddle).
        join(", ") +
        "]" and [" + items.slice(iMiddle).join(", ") + "]</pre>");

    let left = mergeSort(items.slice(0, iMiddle)); ❸
    let right = mergeSort(items.slice(iMiddle));

    // BASE CASE - Returned merged, sorted data:
    // At this point, left should be sorted and right should be
    // sorted. We can merge them into a single sorted list.
    let sortedResult = [];
    let iLeft = 0;

```